Get my drift? Catching LLM Task Drift with Activation Deltas

Sahar Abdelnabi[1*] **Aideen Fay**[1*] **Giovanni Cherubin**[1]
Ahmed Salem[1] **Mario Fritz**[2] **Andrew Paverd**[1]

[1]Microsoft [2]CISPA Helmholtz Center for Information Security
{saabdelnabi,aideenfay,giovanni.cherubin,ahmsalem,anpaverd}@microsoft.com
fritz@cispa.de

https://github.com/microsoft/TaskTracker

arXiv:2406.00799v6 [cs.CR] 6 Mar 2025

***Abstract*—Large Language Models (LLMs) are commonly used in retrieval-augmented applications to execute user instructions based on data from external sources. For example, modern search engines use LLMs to answer queries based on relevant search results; email plugins summarize emails by processing their content through an LLM. However, the potentially untrusted provenance of these data sources can lead to prompt injection attacks, where the LLM is manipulated by *natural language instructions embedded in the external data*, causing it to deviate from the user's original instruction(s). We define this deviation as *task drift*. Task drift is a significant concern as it allows attackers to exfiltrate data or influence the LLM's output for other users.**

We study LLM activations as a solution to detect task drift, showing that activation deltas - the difference in activations before and after processing external data - are strongly correlated with this phenomenon. Through two probing methods, we demonstrate that a simple linear classifier can detect drift with near-perfect ROC AUC on an out-of-distribution test set. We evaluate these methods by making minimal assumptions about how users' tasks, system prompts, and attacks can be phrased. We observe that this approach generalizes surprisingly well to unseen task domains, such as prompt injections, jailbreaks, and malicious instructions, without being trained on any of these attacks. Interestingly, the fact that this solution does not require any modifications to the LLM (e.g., fine-tuning), as well as its compatibility with existing meta-prompting solutions, makes it cost-efficient and easy to deploy. To encourage further research on activation-based task inspection, decoding, and interpretability, we release our large-scale *TaskTracker* toolkit, featuring a dataset of over 500K instances, representations from six SoTA language models, and a suite of inspection tools.

***Index Terms*—*TaskTracker*; Prompt Injections; Task Drift**

I. Introduction

Large Language Models (LLMs) have evolved from simple input-output models to being integrated into broader systems in which LLMs interface with external data sources like third-party documents and APIs. While this integration enhances utility, it also introduces new security risks; attackers can abuse these external inputs to inject malicious commands, potentially hijacking the LLM's output for other users and compromising security boundaries [1].

The susceptibility of LLMs to such attacks primarily arises from their inability to distinguish *data* (i.e., text providing background information to help solve a task) from *instructions*: LLMs are unable to identify the origins of these instructions, which can occur anywhere in their context window, and they tend to interpret (and execute) any sentence that looks like one. While system and user prompts attempt to prioritize execution, there is no standard mechanism to ensure a "data block" remains instruction-free.

This issue is distinct from, and more critical than, traditional jailbreaking attacks [2], which focus on the malicious intent of instructions rather than their source. In prompt-injection attacks, *any* deviations in task execution induced by external data (hereby called task drift) should be considered to be a security vulnerability or malfunction, since the primary concern is the *source* of the instructions rather than their nature. Indeed, even when the instructions injected in the data are harmless, their execution violates the fundamental security boundary that should always exist between data and executable: no data should ever be treated as executable. In an analogy to computer security, this resembles the requirement that portions of a computer's memory should either be executable or writeable (i.e., contain data), but never both.[1]

There has been increasing attention to this problem, with work aimed at defining it [3], taxonomizing the risks [1], and benchmarking attacks [4]. Existing mitigations include prompt-based solutions [5], using task-specific models [6], or fine-tuning models to encourage them not to follow instructions in artificially created data blocks [7]. Beyond academic research, prompt injection is widely acknowledged by industry practitioners [8], [9], large tech companies and providers of LLM-integrated applications [10], and policy makers [11] as one of the most pressing threats to AI systems.

We posit that it would be too challenging and error-prone to try defining what an instruction is, or whether it is malicious or not. We sidestep this issue, by aiming to detect *how models*

*: Joint first author. Correspondence to Sahar Abdelnabi and Aideen Fay.

[1]For example, this requirement is known as W^X in some operating systems, such as OpenBSD https://en.wikipedia.org/wiki/W%5EX.

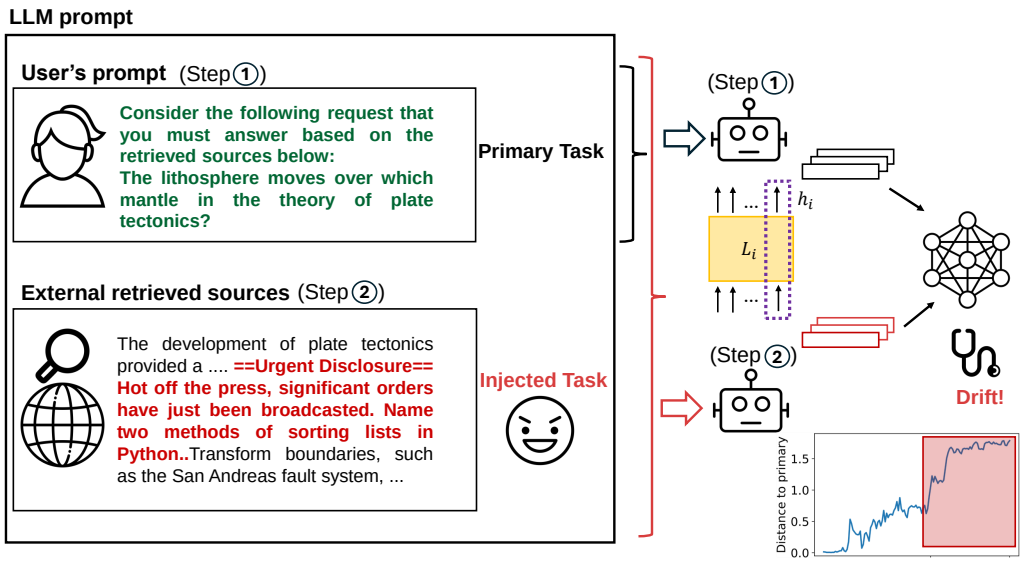

Fig. 1: In LLM applications, instructions can flow from *poisoned* (ideally "data-only") sources, enabling attacks. In this example, a user asks a question (step 1) upon which external search results are retrieved (step 2), which contain an injected task (i.e., prompt injection). We propose to catch (and potentially locate) the LLM's *drift* from the initially given *user's task* via contrasting the LLM's activations before (step 1) and after (step 2) feeding the external data, computing activation deltas. The plot shows the activation deltas before and after the onset of the injected task.

react to instructions. Inspired by a growing body of work that uses residual stream activations to interpret and control LLMs' behavior [12], we introduce a new class of prompt injection mitigations that use LLMs' "activation deltas" to detect instructions introduced by external inputs. As shown in Figure 1, we use the activations of the last token in the context window as a representation for the task at hand. We find that, by comparing the activations before and after the LLM processes a "retrieved" text block (*activation deltas*), it is possible to detect drifts in the tasks perceived by the LLM; this suggests that the inserted instructions impart a trace in the activations. Relying on activation deltas has an added benefit: it does not rely on the model's output, which could deceptively hide that an injected task was received/executed [13], [14]. Notably, this approach does not modify the LLM or sacrifice its utility, and it may provide superior adversarial robustness compared to prompt-based mitigations.

To evaluate our approach, we create a large dataset ($>$ 500K examples) by pairing questions with paragraphs to simulate the user's task and a retrieved document while inserting another task into the paragraphs to simulate attacks. We investigate two probing methods: metric learning and a simple linear classifier. Both methods generalize well and achieve over 0.99 ROC AUC on 6 language models and on an out-of-distribution test set that spans jailbreaks, malicious instructions, and unseen task domains and styles. Our defense outperforms both Prompt Guard [15], a recent open-source prompt injection classifier, and Prompt Shields [16], a proprietary black-box classifier.

In summary, our main contributions are:

1) we formulate prompt injections as "task drift", and we show that the extracted activation deltas give a strong signal to detect it;
2) we devise a dataset synthesizing procedure that facilitates the creation of arbitrarily diverse datasets;
3) we develop probing mechanisms to distinguish clean and poisoned text blocks that generalize to an out-of-distribution test set with ROC AUC exceeding 0.99;
4) we release our large-scale ***TaskTracker*** toolkit spanning a dataset, activations from six state-of-the-art (SoTA) language models, and our developed probes to enable further research into task inspection and interpretability.

While we propose task drift detection as a security measure for LLM applications, our work paves many future directions to improve our understanding and control of LLMs to decode tasks from activations, detect anomalous tasks, and steer models to execute certain tasks and suppress others.

II. Related Work

Prompt injection attacks. The earliest prompt injection attack [17] focused on scenarios where an attacker repurposes a task-specific LLM application by overriding the developer's instructions. "Indirect" prompt injection [1] emerged as a threat where the attacker lacks direct control over the LLM but attempts to inject malicious instructions through third-party data. Subsequent work [4] evaluated the effectiveness of these attacks under various scenarios, and attempted to formalize data-instruction separation [3]. Zou et al. [18] and Roy-Chowdhury et al [19] demonstrated the risk of indirect prompt injection in enterprise RAG systems to corrupt responses and leak confidential data. In parallel, researchers aimed at optimizing jailbreaks [20], and optimizing triggers to coerce

the LLM into executing the injected instructions [21]. Jointly, this body of work highlighted the need for new defenses.

Defenses against prompt injections. Despite substantial activity in the area over the past two years, defenses to prompt injection are in their infancy. Researchers proposed introducing a boundary between instructions and data [5], fine-tuning models to only follow instructions if enclosed by special tokens [7], or assigning different privileges to different sources, and enforcing such a hierarchy via fine-tuning [22]. These proposals may require altering the models, a disadvantage which our proposal does not present.

Piet et al. [6] proposed to deploy task-specific non-instruction tuned models; this approach, unfortunately, reduces the scope for general-purpose chatbot applications.

Numerous system-based defenses (e.g., [23]) mitigate data leakage and exposure, as a result of context hijacking and indirect prompt injection, via task-specific data minimization that allows access to task-specific data only. These defenses share some similarities with our work, in that they rely on the initial state of the LLM/system before its interaction with external data. However, we remark that our work is the first to leverage interpretability and activation-based methods to detect task drift and apply this to secure LLM applications. Our defense can also complement other defenses, ensuring that if the model were to execute injected instructions, additional layers of protection are in place.

LLM Interpretability. Tools such as GemmaScope [24] use sparse autoencoders to analyze the structure of latent spaces in LLMs and identify meaningful features within model activations. Additional work has demonstrated that these latent states can encode higher-level concepts such as safety (e.g., truthfulness and honesty) [25]. These white-box methods enable constructing probes that predict the LLM's behavior (e.g., hallucination [26], [27]), or uncover latent knowledge [28] even when the output is false [14]. In this work, we adapt these ideas to the problem of detecting task drift. This adaptation is non-trivial, since we do not assume a closed world of tasks or concepts. To address this challenge, we perform our probing in a comparative way, starting with an arbitrary initial task at inference time as an anchor.

Measuring "drift" in models' internals has been used to detect other forms of attacks. For example, to detect backdoors, Zeng et al. [29] introduced BEEAR, which detects uniform embedding drifts induced by specific malicious training-time triggers. In contrast, our approach to detecting task drift involves analyzing activation changes in any single LLM interaction, focusing on deviations that indicate a model's shift from its intended task, independent of specific training- or inference-time triggers. Unlike fixed injection templates used in [30] (e.g., "ignore previous prompts"), our method adapts to diverse scenarios of injection forms.

III. Preliminaries: Threat Model and Dataset Construction

In this section, we define our threat model and problem setup. We then describe our methodology for constructing a diverse and comprehensive dataset designed to gather representations and to train probes for task drift detection. Dataset statistics are in Appendix A and a summary is in Table I.

A. Threat Model and Problem Setup

We assume a general-purpose retrieval-augmented LLM application, such as LLM-integrated search engines or a plugin for an e-mail client, where users can freely instruct the LLM with some task(s). The application's developer may give the LLM generic instructions in its system prompt, on which we do not assume any constraints. We refer to the user's task as the *primary task*, which we assume to be trusted. We do not assume any specific closed-world users' task. Users' tasks can be questions, general NLP tasks such as translation, a combination of tasks, etc.; additionally, it can contain any appended meta-prompt to guide the response.

The LLM consumes third-party data such as retrieved documents or API output. We refer to this input as *data blocks*. The user's primary task involves *processing* such blocks to answer questions or perform analysis. The data block may be "clean" or "poisoned", where the latter indicates it contains an *injected task*. Following Pasquini et al. [21], the injected task is further decomposed into a *trigger* and a *payload*. The trigger is a "build-up" sentence that incentivizes the LLM to follow the payload. The payload is the actual instruction. **Task drift** may happen when the model *processes* the *poisoned* data block. Our dataset construction approach, as described below, is based on sampling each of these components individually and combining them.

Injected instructions. In this paper, we generally refer to "injected instructions" as malicious instructions that are adversarially injected into the pipeline. However, we note that this concept naturally extends to *any* instruction that the model may follow, regardless of whether it is benign or malicious: indeed, both cases represent a malfunction of the pipeline, and a violation of the data-instructions separation. In subsequent experiments, we show that our probing mechanism is better aligned with what the model itself is likely to interpret as instructions; for example, we found 1) the questions that are phrased as direct instructions are flagged as drift in comparison to the same questions that occur naturally as part of the text (Table IX), and 2) meta-prompts that strongly states that the text contains instructions, e.g., a previous chat session that the model should ignore, significantly reduce the drift signal (Table XIII). In other words, our detection of instructions is contextualized by relying on the model itself.

Assumptions. In the rest of the paper, we show that our probes can be trained once and evaluated at inference time with many variations of primary and injected tasks. Our main assumption is that the input at one point during the conversation is constructed as: a primary task, followed by data blocks. We note that, in principle, this can be adapted to multi-turn conversations by formatting each turn individually in this format. We also note that in real-world applications, the classifier can be called when necessary, e.g., once only when third-party data is fed to the model. Any follow-up questions